题目 **1**：

请完成以下操作：

1. 使用切片取出 `"Pyt"` 和 `"123"`。
2. 使用 `for`，。
3. 将提取出的数字转成整数，并格式化为 左对齐、宽度 10 的形式。（输出示例：`'123'`）

```
s = "Python123"

# 1. 使用切片取出`"Pyt"` 和`"123"`
s1 = s[:3]
s2 = s[-3:]
print(s1, s2)

# 2. 使用`for `循环提取其中的数字字符，并拼接成新的字符串。
s3 = ""
for i in s:
    if ord('0') <= ord(i) <= ord('9'):
        s3 = s3 + i

# 3. 将提取出的数字转成整数，并格式化为 左对齐、宽度10 的形式。（输出示例：
`'123'`）
num = int(s3)
print(f'{num:<10d}')


Pyt 123
123
```

题目 **2**：

请完成以下操作：

1. 使用 3 种方式 输出 `"Alice's score is 87"（%d、format()、f-string）`。
2. 使用 `replace()` 将 `"Alice"` 替换为 `"Bob"`。
3. 将 GPA 保留两位小数并输出（例如：`3.14`）。
4. 将 GPA 输出为 百分比格式，保留两位小数。（示例：`314.16%`）

```
name = "Alice"
score = 87
gpa = 3.1415926

# 1. 使用3 种方式 输出`"Alice's score is 87"（%d、format()、f-string）`。
```

```
print(name + "'s score is %d" % score)
print(name + "'s score is {0}".format(score))
print(name + f"'s score is {score}")

# 2. 使用`replace()` 将`"Alice"` 替换为`"Bob"`。
name = name.replace("Alice", "Bob")
print(name)

# 3. 将GPA （:`3.14`）。
print(name + f"'s GPA is {gpa:.2f}")

# 4. 将GPA 输出为百分比格式，保留两位小数。（示例：`314.16%`）
print(name + f"'s GPA is {gpa:.2%}")


Alice's score is 87
Alice's score is 87
Alice's score is 87
Bob
Bob's GPA is 3.14
Bob's GPA is 314.16%
```

题目 **3**：

请完成以下操作：

1. 使用 `split()` 。
2. 找出 `"Python"`的索引位置。
3. 使用 `join()` 拼接为 `"I-love-Python-very-much"`。
4. 统计 `"o"` 出现次数，并输出：

- `"The letter o appears 3 times."`
- 要求：用 f-string + （, 1,000）。

```
text = "I,love,Python,very,much"

# 1. 使用`split()` 拆分为列表。
text_list = text.split(",")
print(text_list)

# 2. 找出`"Python" `的索引位置。
print(f"\"Python\"的索引位置：{text.find("Python")}")

# 3. 使用`join()` 拼接为`"I-love-Python-very-much"`。
print("-".join(text_list))

'''
4. 统计"o" 出现次数，并输出：
```

```
    "The letter o appears 3 times."
    要求：用f-string + 千分位格式（如果次数很大，也能显示为1,000 这种形式）。
'''
print(f"The letter o appears {text.count('o'):,} times.")
```

```
['I', 'love', 'Python', 'very', 'much']
"Python"的索引位置：7
I-love-Python-very-much
The letter o appears 2 times.
```

题目1：

给定两个班级本学期选课名单（学号字符串）：

请完成：

1. 分别求出同时选课的学生、仅在 A 的学生、仅在 B 的学生、至少选了一门课的学生。（要求使用集合运算符：`&`、`-`、`^`、`|`）
2. 判断 `{'S02','S05'}` 是否为 A 的子集；判断 A 与 `{'X01','X02'}` 是否不相交。
3. 新增转入学生：`'S11','S12'` 到集合 A；一次性把 `['S06','S07']` 并入集合 A。
4. 然后尝试用 `remove` 和 `discard` 分别删除 `'S07'` 与 `'S100'`，观察两者行为差异并。
5. 将“至少选了一门课的学生”按学号从小到大输出为列表；再用集合推导式构造一个新集合，保留学号末位为偶数的学生。

```
A = {'S01', 'S02', 'S03', 'S05', 'S08', 'S10'}
B = {'S02', 'S04', 'S05', 'S06', 'S09', 'S10'}

# 1. 分别求出同时选课的学生、仅在A 的学生、仅在B 的学生、至少选了一门课的学生。
（要求使用集合运算符：`&、-、^、|`）
print(f"同时选课的学生：{A & B}")
print(f"仅在A的学生：{A - B}")
print(f"仅在B的学生：{B - A}")
C = A | B
print(f"至少选了一门课的学生：{A | B}")

# 2. 判断`{'S02','S05'}` 是否为A 的子集；判断A 与`{'X01','X02'}` 是否不相
交。
print("{'S02','S05'} A 的子集：" + str({'S02', 'S05'} <= A))
print(" A 与{'X01','X02'} 是否不相交：" + str({'X01',
'X02'}.isdisjoint(A)))

# 3. 新增转入学生：`'S11','S12'` 到集合A；`['S06','S07']` 并入集合A。
A.update({'S11', 'S12'})
A |= {'S06', 'S07'}
```

```python
print(f'新增后的A：{A}')

# 4. 然后尝试用` remove` 和`discard` 分别删除``S07`` 与``S100``，观察两者行为差异并写下结论。
print('remove和discard的区别：remove在删除不存在的元素时会抛出KeyError，discard。')

# 5. 将“至少选了一门课的学生”按学号从小到大输出为列表；再用集合推导式构造一个新集合，保留学号末位为偶数的学生。
C = list(sorted(C))
print(f"至少选了一门课的学生：{C}")
D = {i for i in C if (int(i[-1]) % 2 == 0) & (i[-1] != '0')}
print(f"学号末位为偶数的学生：{D}")
```

```
同时选课的学生：{'S05', 'S10', 'S02'}
仅在A的学生：{'S01', 'S03', 'S08'}
仅在B的学生：{'S09', 'S06', 'S04'}
至少选了一门课的学生：{'S08', 'S06', 'S03', 'S09', 'S04', 'S01', 'S10', 'S05', 'S02'}
{'S02','S05'}是否为A 的子集：True
 A 与{'X01','X02'} 是否不相交：True
新增后的A：{'S12', 'S08', 'S06', 'S03', 'S01', 'S11', 'S10', 'S05', 'S02', 'S07'}
remove和discard的区别：remove在删除不存在的元素时会抛出KeyError，discard则什么都不做。
至少选了一门课的学生：['S01', 'S02', 'S03', 'S04', 'S05', 'S06', 'S08', 'S09', 'S10']
学号末位为偶数的学生：{'S06', 'S02', 'S04', 'S08'}
```

题目 2:

给定三条评论：

请完成：

1. 对每条评论按空格切分成单词列表；用集合去重得到“每条评论的唯一单词集合”。
2. 用列表推导式与集合推导式各实现一次（两种写法）。
3. 构建“全局词频字典” `freq`，键为单词、值为出现总次数。
4. 不使用外部库；要求用 `dict.get(k, 0)` 或 `setdefault` 两种方式分别实现（得到等价结果）。
5. 构建“倒排索引” index：键为单词，值为 出现该词的文档编号集合（编号从 0 开始）。

- 提示：`index.setdefault(word, set()).add(doc_id)`

1. 输出出现次数最多的 3 个单词（按次数降序；如并列，按单词字母序升序）。给出结果列表形如 [('python', 3), ('data', 3), ('and', 2)]。

2. 把 `index` 反转为 `by_doc_words`（键为 `doc_id`，值为该文档出现的单词按字母序），。

```
docs = [
    "data mining with python and sql",
    "python for data analysis and visualization",
    "mining patterns in data using python"
]

# 1. 对每条评论按空格切分成单词列表；用集合去重得到“每条评论的唯一单词集合”。
# 2. 用列表推导式与集合推导式各实现一次（两种写法）。
words_list = [doc.split(" ") for doc in docs]
unique_words_list1 = [set(words) for words in words_list]
unique_words_list2 = [{word for word in words} for words in
words_list]
print(unique_words_list1)
print(unique_words_list2)

# 3. 构建“全局词频字典”`freq`，键为单词、值为出现总次数。
# 4. 不使用外部库；要求用`dict.get(k, 0)` 或`setdefault` 两种方式分别实现（得到等价结果）。
words_list_all = [word for words in words_list for word in words]
word_count1 = {}
word_count2 = {}
for word in words_list_all:
    word_count1[word] = word_count1.get(word, 0) + 1
for word in words_list_all:
    word_count2[word] = word_count2.setdefault(word, 0) + 1
print(word_count1)
print(word_count2)

# 5. 构建“倒排索引”index：键为单词，值为 出现该词的文档编号集合（编号从0 开始）。
inverted_index = {}
for doc_id, doc in enumerate(unique_words_list1):
    for word in doc:
        inverted_index.setdefault(word, set()).add(doc_id)
print(inverted_index)

# 6. 输出出现次数最多的3 （;，）。[('python', 3), ('data', 3), ('and',
2)]。
word_count_most = [item for item in sorted(word_count1.items(),
key=lambda x: (-x[1], x[0]))[:3]]
print(word_count_most)

# 7. `index` 反转为`by_doc_words`（键为`doc_id`，值为该文档出现的单词按字母序排序后的元组），要求用字典推导式完成。
by_doc_words = {doc_id: tuple(sorted([word for word, ids in
inverted_index.items() if doc_id in ids])) for doc_id in
                {doc_id for docs_id in inverted_index.values() for
doc_id in docs_id}}
```

```
print(by_doc_words)
```

```
[{'python', 'with', 'data', 'and', 'mining', 'sql'}, {'python',
'data', 'analysis', 'and', 'for', 'visualization'}, {'python', 'data',
'mining', 'in', 'using', 'patterns'}]
[{'python', 'with', 'data', 'and', 'mining', 'sql'}, {'python',
'data', 'analysis', 'and', 'for', 'visualization'}, {'python', 'data',
'mining', 'in', 'using', 'patterns'}]
{'data': 3, 'mining': 2, 'with': 1, 'python': 3, 'and': 2, 'sql': 1,
'for': 1, 'analysis': 1, 'visualization': 1, 'patterns': 1, 'in': 1,
'using': 1}
{'data': 3, 'mining': 2, 'with': 1, 'python': 3, 'and': 2, 'sql': 1,
'for': 1, 'analysis': 1, 'visualization': 1, 'patterns': 1, 'in': 1,
'using': 1}
{'python': {0, 1, 2}, 'with': {0}, 'data': {0, 1, 2}, 'and': {0, 1},
'mining': {0, 2}, 'sql': {0}, 'analysis': {1}, 'for': {1},
'visualization': {1}, 'in': {2}, 'using': {2}, 'patterns': {2}}
[('data', 3), ('python', 3), ('and', 2)]
{0: ('and', 'data', 'mining', 'python', 'sql', 'with'), 1:
('analysis', 'and', 'data', 'for', 'python', 'visualization'), 2:
('data', 'in', 'mining', 'patterns', 'python', 'using')}
```

题目 3:

请完成：

1. 分别求出两家都卖的商品、只在 A 的商品、只在 B 的商品。（提示：对 `keys()` 做集合运算）
2. 生成一个新字典 `merged`，包含两家所有商品的价格。若同一商品两家都卖，取 更 。

- 要求：只使用内置字典与集合操作（可以使用推导式、`min`、`update`），。
1. 生成一个字典 `diff`：仅包含两家都卖但价格不同的商品，值为“绝对价差”。结果按“价差”，。

```
store_A = {'apple': 3.5, 'banana': 2.0, 'milk': 6.8, 'bread': 7.2}
store_B = {'banana': 2.2, 'milk': 6.5, 'bread': 7.2, 'yogurt': 5.0}

# 1. 分别求出两家都卖的商品、只在A 的商品、只在B 的商品。
A_keys = store_A.keys()
B_keys = store_B.keys()
print(f"两家都卖的商品：{A_keys & B_keys}")
print(f"只在A 的商品：{A_keys - B_keys}")
print(f"只在B 的商品：{B_keys - A_keys}")

# 2. 生成一个新字典`merged`，包含两家所有商品的价格。若同一商品两家都卖，取 更低价格。
merged1 = {goods: min([price for key, price in store_A.items() |
```

```
store_B.items() if goods == key]) for goods in
           A_keys | B_keys}
merged2 = {goods: min(store_A.get(goods, float("inf")),
store_B.get(goods, float("inf"))) for goods in
           A_keys | B_keys}
print(merged1)
print(merged2)

# 3. 生成一个字典`diff`：仅包含两家都卖但价格不同的商品，值为“绝对价差”。结果按“价差从大到小，商品名字母序次序”输出为列表。
diff = {goods: abs(round(store_A[goods] - store_B[goods], 1)) for
goods in A_keys & B_keys if
        store_A[goods] != store_B[goods]}
print(sorted(diff.items(), key=lambda x: (-x[1], x[0])))
```

```
两家都卖的商品：{'milk', 'banana', 'bread'}
只在 A 的商品：{'apple'}
只在 B 的商品：{'yogurt'}
{'bread': 7.2, 'banana': 2.0, 'apple': 3.5, 'milk': 6.5, 'yogurt':
5.0}
{'bread': 7.2, 'banana': 2.0, 'apple': 3.5, 'milk': 6.5, 'yogurt':
5.0}
[('milk', 0.3), ('banana', 0.2)]
```

题目 4:

有若干次一起购买的商品集合（同一次购买中同一商品不会重复）：

```
baskets = [
    {'apple','banana','milk'},
    {'banana','bread'},
    {'milk','bread','banana'},
    {'yogurt','banana'},
    {'apple','milk'}
]
```

请完成：

1. “ ”（ `{apple, banana}` 与 `{banana, apple}` ）。
- 要求：用 `frozenset` 表示无序对；同一篮子内不要重复计数。
1. 构建字典 `pair_count`：键为 `frozenset({'item1','item2'})`，值为该对在全部篮子中出现的次数。
- 只能使用内置工具；建议 `pair_count[pair] = pair_count.get(pair, 0) + 1`
1. 找出出现次数最多的前 3 个无序对，并输出为形如 `[({'banana','milk'}, 2), ...]` 的列表。（集合显示顺序不作要求）

2. 给定一个新篮子 `new_basket = {'banana','milk','bread'}`，判断它与只含 `'yogurt'` 的篮子在商品集合上是否不相交；同时验证 `{'banana','milk'} ⊆ new_basket` 是否成立。

```python
baskets = [
    {'apple', 'banana', 'milk'},
    {'banana', 'bread'},
    {'milk', 'bread', 'banana'},
    {'yogurt', 'banana'},
    {'apple', 'milk'}
]

# 1. 对每个篮子，计算其中所有“无序商品对”（例如`{apple, banana}` 与`{banana, apple}` 视为同一对）。
disorder_pairs = {
    no: {frozenset([list(basket)[i], list(basket)[j]]) for i in range(len(basket)) for j in range(i + 1, len(basket))}
    for no, basket in enumerate(baskets)}
for value in disorder_pairs.values():
    print(value)

# 2. 构建字典`pair_count`：键为`frozenset({'item1','item2'})`，值为该对在全部篮子中出现的次数。
pair_count = {}
for value in disorder_pairs.values():
    for pair in value:
        pair_count[pair] = pair_count.get(pair, 0) + 1
print(pair_count)

# 3. 找出出现次数最多的前3 个无序对，并输出为形如`[({'banana','milk'}, 2), ...]` 的列表。（集合显示顺序不作要求）
pair_most = [(set(item[0]), item[1]) for item in sorted(pair_count.items(), key=lambda x: -x[1])[:3]]
print(pair_most)

# 4. 给定一个新篮子`new_basket = {'banana','milk','bread'}`，判断它与只含`'yogurt'` 的篮子在商品集合上是否不相交；同时验证`{'banana','milk'} ⊆ new_basket` 是否成立。
new_basket = {'banana', 'milk', 'bread'}
print(
    f"新篮子与只含 'yogurt' 的篮子在商品集合上是否不相交：{new_basket.isdisjoint([basket for basket in baskets if 'yogurt' in basket][0])}")
print(f"{{'banana','milk'}} ⊆ new_basket 是否成立：{ {'banana', 'milk'} <= new_basket}")
```

```
{frozenset({'apple', 'milk'}), frozenset({'milk', 'banana'}), frozenset({'apple', 'banana'})}
{frozenset({'bread', 'banana'})}
{frozenset({'milk', 'bread'}), frozenset({'bread', 'banana'}), frozenset({'milk', 'banana'})}
{frozenset({'banana', 'yogurt'})}
{frozenset({'apple', 'milk'})}
{frozenset({'apple', 'milk'}): 2, frozenset({'milk', 'banana'}): 2,
```

```
frozenset({'apple', 'banana'}): 1, frozenset({'bread', 'banana'}): 2,
frozenset({'milk', 'bread'}): 1, frozenset({'banana', 'yogurt'}): 1}
[({'apple', 'milk'}, 2), ({'milk', 'banana'}, 2), ({'bread',
'banana'}, 2)]
新篮子与只含 'yogurt' 的篮子在商品集合上是否不相交：False
{'banana','milk'} ⊆ new_basket 是否成立：True
```

题目 1：小型文本工具模块 `text_tools_easy.py`

- 要求 在同目录新建文件 **`text_tools_easy.py`**，实现下列函数：

1. `clean_text(s: str, lower: bool = True) -> str`
   - ：；；。
2. `tokens(s: str) -> list[str]`
   - 功能：按空格切分，返回词列表。
3. `word_count(tokens: list[str]) -> dict[str, int]`
   - 功能：词频统计。
4. `topk(freq: dict[str, int], k: int = 5) -> list[tuple[str,int]]`
   - 功能：按“次数降序、词典序升序”返回前 k 。

期望形如：[('data', 2), ('mining', 1), ('python', 1)]

```python
from typing import List, Dict, Tuple


def clean_text(s: str, lower: bool = True) -> str:
    s = s.strip()
    if lower:
        s = s.lower()
    s = " ".join(s.split())
    return s


def tokens(s: str) -> List[str]:
    return s.split()


def word_count(toks: List[str]) -> Dict[str, int]:
    freq = {}
    for tok in toks:
        freq[tok] = freq.get(tok, 0) + 1
    return freq


def topk(freq: Dict[str, int], k: int = 5) -> List[Tuple[str, int]]:
    return sorted(freq.items(), key=lambda x: (-x[1], x[0]))[:k]


s = " Data   mining with  Python and data   "
s = clean_text(s)  # "data mining with python and data"
```

```python
ts = tokens(s)  # ['data','mining',...]
freq = word_count(ts)
print(topk(freq, k=3))
```

```
[('data', 2), ('and', 1), ('mining', 1)]
```